

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Расстояние Левенштейна
Вариант 11

Студент гр. 4343

Васильев А. С.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы

Изучить алгоритм вычисления редакционного расстояния и модифицировать его для решения индивидуального варианта №11: среди всех префиксов и суффиксов первой строки найти те, которые имеют минимальное редакционное расстояние до второй строки, вывести найденное минимальное расстояние и соответствующие префиксы и суффиксы. При этом необходимо сохранить асимптотическую сложность не хуже, чем у стандартного алгоритма вычисления редакционного расстояния, то есть $O(nm)$, где n и m — длины сравниваемых строк.

Задание

Среди всех префиксов первой строки найти те, редакционное расстояние от которых до второй строки минимально, вывести это расстояние и эти префиксы. Среди всех суффиксов первой строки найти те, редакционное расстояние от которых до второй строки минимально, вывести это расстояние и эти суффиксы. Асимптотическая сложность решения задачи должна быть не больше, чем при обычном вычислении редакционного расстояния.

Описание алгоритма

В данной работе используется алгоритм вычисления редакционного расстояния, основанный на динамическом программировании. Редакционное расстояние показывает минимальное количество операций, необходимых для преобразования одной строки в другую. В рассматриваемой задаче используются три операции: вставка символа, удаление символа и замена одного символа другим.

Пусть заданы две строки A и B , где n — длина строки A , а m — длина строки B . Для вычисления редакционного расстояния вводится динамическое состояние $dp[i][j]$.

Значение $dp[i][j]$ означает минимальное количество операций, необходимых для преобразования первых i символов строки A в первые j символов строки B .

Если один из рассматриваемых префиксов пустой, значение таблицы определяется напрямую. Для преобразования первых i символов строки A в пустую строку необходимо удалить все эти символы, поэтому $dp[i][0] = i$.

Для преобразования пустой строки в первые j символов строки B необходимо вставить все символы строки $B[:j]$, следовательно $dp[0][j] = j$.

Для остальных ячеек таблицы рассматриваются три варианта перехода:

- $dp[i - 1][j] + 1$ — удаление символа из первой строки,
- $dp[i][j - 1] + 1$ — вставка символа,
- $dp[i - 1][j - 1] + cost$ — замена или совпадение символа.

Если символы $A[i - 1]$ и $B[j - 1]$ совпадают, то $cost = 0$, так как дополнительная операция не требуется. Если символы различаются, то $cost = 1$, что соответствует операции замены.

По заданию требуется найти не только расстояние между полной первой строкой и второй строкой, но и определить, какие префиксы и суффиксы первой строки имеют минимальное редакционное расстояние до второй строки.

Для поиска префиксов используется прямой проход динамического программирования. После обработки префикса $A[:i]$ значение последнего элемента текущей строки DP соответствует расстоянию между этим префиксом и всей строкой B. Поэтому после одного прохода можно получить расстояния для всех префиксов строки A.

Для поиска суффиксов используется тот же алгоритм динамического программирования, что и для префиксов, но применяется он к развёрнутым строкам $A[::-1]$ и $B[::-1]$. В этом случае каждый префикс развёрнутой строки соответствует некоторому суффиксу исходной строки. После вычисления расстояний для префиксов развёрнутой строки значения переносятся обратно: расстояние для суффикса $A[i:]$ берётся из результата для префикса длины $\text{len}(A) - i$ развёрнутой строки. Таким образом, отдельный обратный проход справа налево не требуется, так как задача для суффиксов сводится к уже реализованному вычислению расстояний для префиксов.

Сложность алгоритма

Обычный алгоритм вычисления редакционного расстояния для строк длины n и m работает за $O(nm)$. В данной модификации выполняются два прохода динамического программирования: один для префиксов и один для суффиксов. Каждый проход имеет сложность $O(nm)$, поэтому итоговая сложность остаётся $O(nm)$. Память используется $O(n + m)$, так как хранятся только текущая строка DP и массив расстояний для найденных префиксов и суффиксов.

Тестирование

Входные данные	Результат
abc abc	PREFIXES 0 abc SUFFIXES 0 abc
abc “”	PREFIXES 0 “” SUFFIXES 0 “”
“” abc	PREFIXES 3 “” SUFFIXES 3 “”
abc def	PREFIXES 3 “” a ab abc SUFFIXES 3 abc bc c “”
abcdef abc	PREFIXES 0 abc SUFFIXES 3 abcdef def ef f “”